2026-5-14 C++vector--底层实现

- 1.默认成员函数
  - 1.1构造函数
    - 1.1.1 初始化列表构造
    - 1.1.2 任意多参数构造：
    - 1.1.3 迭代器区间构造:
  - 1.2 析构函数
  - 1.3 拷贝构造函数
  - 1.4运算符号重载
    - 1.4.1下标访问运算符重载
  - 1.5 赋值运算符重载
- 2.迭代器
- 3. 修改器
  - 3.1push_back尾插
  - 3.2insert插入函数
  - 3.3 erase去除函数
- 4.容量和空间相关的
  - 4.1 reserve扩容
  - 4.2 size函数
  - 4.3 capacity函数
  - 4.4 resize函数
  - 4.5 clear函数
  - 4.6 empty函数

1.默认成员函数

1.1构造函数

1.1.1 初始化列表构造

```
vector()
    :_start(nullptr)
    , _finish(nullptr)
    , _endofstorage(nullptr)
{
```

```
}
```

关于上述用初始化列表构造，如果我们在定义其私有成员变量的时候给定初始值的话，列表就可以省略：

```
vector(){}
```

但是这个东西可以省略吗？答案是不可以的，不写程序不会去调用默认构造函数，因为函数调用默认构造函数的规则是：如果函数没有构造，那就调用默认成员函数，其中拷贝构造实际上也是构造没有这个编译器会调用拷贝构造进行构造会造成构造参数不匹配。
或者说我们可以：

```
vector()=default;
```

不同编译器对内置类型的处理机制可能是不一样的。

1.1.2 任意多参数构造：

```
//ZL::vector<int> v1 = { 1,2,3,4,5,6 };
vector(initializer_list<T> i1) {
    reserve(i1.size());
    for (auto& e : i1) {
        push_back(e);
    }
}
```

1.1.3 迭代器区间构造:

```
//ZL::vector<int> v3(v1.begin() + 1, v1.begin() +4 );
//函数模板，任意类型容器迭代器初始化
template<class InputIterator>
vector(InputIterator first, InputIterator last) {
    while (first != last) {
        push_back(*first);
        ++first;
```

```
    }
}
```

1.2 析构函数

```
//析构
~vector() {
    if (_start) {
        delete[]_start;
        _start = _finish = _endofstorage = nullptr;
    }
```

1.3 拷贝构造函数

这个的原理是，我们将
v1传递进来，首先开辟一个与v1相同的空间，然后遍历v1将v1的值逐一尾插到v2中。

```
//v2(v1);
vector(const vector<T>& v) {
    reserve(v.capacity());
    for (auto& e : v) {
        push_back(e);//将被拷贝的值遍历并逐一拷贝进去
    }
}
```

1.4运算符号重载

1.4.1下标访问运算符重载

```
T& operator[](size_t i) {
    assert(i < size());
    return _start[i];
}
const T& operator[](size_t i) const{
    assert(i < size());
    return _start[i];
}
```

1.5 赋值运算符重载

```
//这里要补一个交换
void swap(vector<T>& v) {
    std::swap(_start, v._start);
    std::swap(_finish, v._finish);
    std::swap(_endofstorage, v._endofstorage);
}
//赋值
vector<T>& operator=(vector<T> v) {
    swap(v);
    return *this;
}
```

2.迭代器

```
//同样的这里需要定义
using iterator = T*;//using是可以替代typedef的，功能上覆盖typedef
using const_iterator = T*;//但是使用using的优势是using可以为函数模板取别名
/*typedef iterator = T*;
typedef const_iterator = T*;*/
iterator begin() {
    return _start;
}
iterator end() {
    return _finish;
}
const_iterator begin() const {
    return _start;
}
const_iterator end() const {
    return _finish;
}
```

3. 修改器

3.1push_back尾插

```
void push_back(const T& x) {
    if (_finish == _endofstorage) {
        reserve(capacity() == 0 ? 4 : 2 * capacity());
    }
```

```
    *_finish = x;
    ++_finish;
}
```

3.2insert插入函数

```
void insert(iterator pos,const T& x) {
    assert(pos >= _start);
    assert(pos <=_finish);
    if (_finish == _endofstorage) {
        size_t len = pos - _start;
        reserve(capacity() == 0 ? 4 : capacity() * 2);
        pos = _start + len;//这里需要更新pos,否则一旦发生异地扩容，迭代器就会失效
    }
    //数据移动
    iterator end = _finish-1;
    while (end >= pos) {
        *(end + 1) = *end;
        --end;
    }
    //将数据插入
    *pos = x;
    _finish++;
}
```

但是如果我们想要在特定位置输入：

```
void text_vector03() {
    //那现在如果我想输入一个指定的值，在这个值的位置插入呢？
    ZL::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(4);
    Print(v);
    int x;
    cin >> x;
    auto it = find(v.begin(), v.end(), x);//调用的是命名空间里面的查找，std命名空间是展开的
    //这里的it会失效，不能继续使用
    //扩容会导致it失效，但是C++标准没有规定vector容器的扩容规则
    //所以我们认为insert以后，it就失效了不能再使用了
    //这里能不能加引用解决？首先我们库里面的源码没有引用，
```

```
    //其次我们加了引用的话是对begin传值返回的引用进行返回，临时对象具有常性加了应用会出
错
    if (it != v.end()) {
        v.insert(it, x * 10);

    }
    Print(v);
}
```

那么如果是因为扩容而导致的迭代器失效，那么我们在每次扩容的时候更新一下迭代器就可以解决这个问题了。

```
iterator insert(iterator pos,const T& x) {
    assert(pos >= _start);
    assert(pos <=_finish);
    if (_finish == _endofstorage) {
        size_t len = pos - _start;
        reserve(capacity() == 0 ? 4 : capacity() * 2);
        pos = _start + len;//这里需要更新pos,否则一旦发生异地扩容，迭代器就会失效
    }
    //数据移动
    iterator end = _finish-1;
    while (end >= pos) {
        *(end + 1) = *end;
        --end;
    }
    //将数据插入
    *pos = x;
    _finish++;
    return pos;
}
```

3.3 erase去除函数

```vector
std::vector<int> v;
v.push_back(1);
v.push_back(2);
v.push_back(3);
v.push_back(4);
v.push_back(5);
Print(v);
```

```
[root@bite ~]# g++ test.cpp
[root@bite ~]# ./a.out
1 2 3 4 5
1 3 5
```

```vector
std::vector<int> v;
v.push_back(1);
v.push_back(2);
v.push_back(3);
v.push_back(4);
v.push_back(4);
v.push_back(5);
Print(v);
```

```
[root@bite ~]# g++ test.cpp
[root@bite ~]# ./a.out
1 2 3 4 4 5
1 3 4 5
```

```vector
std::vector<int> v;
v.push_back(1);
v.push_back(2);
v.push_back(3);
v.push_back(4);
v.push_back(4);
v.push_back(5);
v.push_back(4);
Print(v);
```

```
[root@bite ~]# g++ test.cpp
[root@bite ~]# ./a.out
1 2 3 4 4 5 4
Segmentation fault
```

```vector
// 要求删除所有偶数
auto it = v.begin();
while (it != v.end())
{
        if (*it % 2 == 0)
        {
                v.erase(it);
        }

        ++it;
}
```

在VS中，it是完全失效的，在gcc下虽然能跑但是该程序存在很大的隐患。我们可以看到当出现连续偶数的时候后面跟的计数会失效，如果插入的最后一个元素是偶数的话也会失效。

```
void erase(iterator pos) {
    assert(pos >= _start);
    assert(pos < _finish);
    //挪动数据
    iterator it = pos + 1;
    while (it != _finish) {
        *(it - 1) = *it;
        ++it;
```

```
    }
    --_finish;
}
```

同样测试it到底能用吗?

```
void text_vector04() {
    //那现在如果我想输入一个指定的值，在这个值的位置插入呢？
    ZL::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(4);
    Print(v);
    v.erase(v.begin() + 1);
    Print(v);
    //如果我想删除某个特定的元素
    int x;
    cin >> x;
    auto it = find(v.begin(), v.end(), x);
    if (it != v.end()) {//问题来了这里的it还能用吗？
        v.erase(it);
    }
    Print(v);
}
```

这个看不出来，我们重新来举一个例子：

```
//根据上述我们来测试一个用例
void text_vector05() {
    ZL::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(4);
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(1);
    Print(v);
    //要求删除所有的偶数
    auto it = v.begin();
    while (it != v.end()) {
```

```
            if (*it % 2 == 0) {
                it = v.erase(it);//vs下的编译器会对其进行严格的检查
                //这里的迭代器同样也失效了，失效的迭代器就不能再使用了
                //那怎么办？既然it失效了那就让其有效嘛！我们库里面是怎么做的？
                //让新的pos作为返回值返回
            }
            else {
                ++it;
            }
        }
        Print(v);
}
```

将pos更新以防止it失效,同样我们需要更新pos：

```
iterator erase(iterator pos) {
    assert(pos >= _start);
    assert(pos < _finish);
    //挪动数据
    iterator it = pos + 1;
    while (it != _finish) {
        *(it - 1) = *it;
        ++it;
    }
    --_finish;
    return pos;
}
```

4.容量和空间相关的

4.1 reserve扩容

```
void reserve(size_t n) {
    size_t old_size = size();
    if (n > capacity()) {
        T* tmp = new T[n];
        //拷贝旧空间到新空间
        if (_start) {
            memcpy(tmp, _start, sizeof(T) * size());
            delete[]_start;
        }
        _start = tmp;
```

```
        _finish = _start + old_size;
        _endofstorage = _start + n;
    }
}
```

上述代码：
`memcpy` 用于非 POD 类型是未定义行为，如 `vector<string>` 会崩，这里只拷贝了 `size()` 个字节，而 `size()` 返回的是元素个数，不是字节数，属于是浅拷贝。我们在这里需要**逐个拷贝构造新元素**。

```
void reserve(size_t n) {
    size_t old_size = size();
    if (n > capacity()) {
        T* tmp = new T[n];
        //拷贝旧空间到新空间
        if (_start) {
            //memcpy(tmp, _start, sizeof(T) * size());
            for (size_t i = 0; i < old_size; i++) {
                tmp[i] = _start[i];
            }
            delete[]_start;
        }
        _start = tmp;
        _finish = _start + old_size;//这里如果不是新的_size就会不对
        _endofstorage = _start + n;
    }
}
```

1. int这样的类型，赋值可以完成浅拷贝
2. string这样的类型，赋值调用operator=完成拷贝

4.2 size函数

```
size_t size()const
{
    return _finish - _start;//还不在一个空间里会出错
}
```

4.3 capacity函数

```
size_t capacity()const
{
    return _endofstorage - _start;
}
```

4.4 resize函数

```
void resize(size_t n, const T& val = T()) {
    if (n < size()) {
        // 缩小：直接截断
        _finish = _start + n;
    } else if (n > size()) {
        // 扩大：先保证容量，再填充
        reserve(n);
        while (_finish != _start + n) {
            *_finish++ = val;
        }
    }
    // n == size() 时，什么也不做
}
```

4.5 clear函数

```
void clear() {
    _finish = _start;
}
```

4.6 empty函数

```
bool empty() {
    return _finish <= _start;
}
```